

Drat, segnalare eventuali imperfezioni

Arduino e Raspberry Pi: concetti base e positioning

LEZIONE 1 – Arduino

1.1 Che cos'è Arduino

Arduino non è un “mini computer”, ma una scheda elettronica programmabile basata su microcontrollore.

Un microcontrollore è un componente che integra nello stesso chip:

- CPU
- memoria limitata
- periferiche di input/output

Arduino è progettato per controllare dispositivi fisici, reagire a segnali elettrici ed eseguire un comportamento specifico. Non è pensato per eseguire applicazioni generiche come un PC.

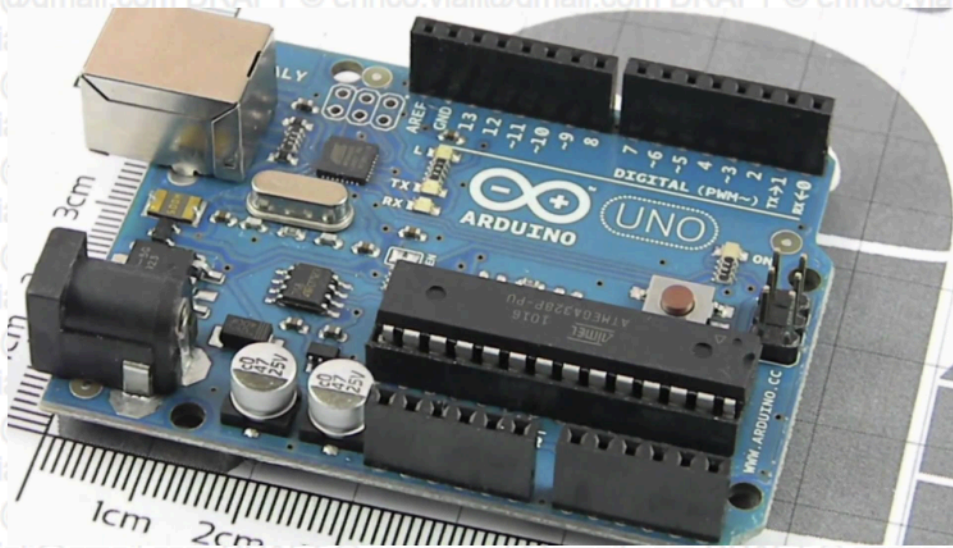


Figure 1: Image

1.2 Differenza rispetto a un PC

Un PC usa un sistema operativo, come Windows o Linux, gestisce più programmi contemporaneamente e interagisce con l'utente tramite mouse, tastiera e interfaccia grafica.

Arduino, invece:

- non ha un sistema operativo
- esegue un solo programma
- lavora in un ciclo continuo

Struttura tipica:

```
void setup() {  
    // inizializzazione  
}  
  
void loop() {  
    // eseguito continuamente  
}
```

Questo modello è diverso da quello di un PC: non ci sono finestre, non c'è multitasking reale e non c'è una gestione avanzata della memoria.

1.3 Perché Arduino esiste

Arduino è stato progettato per controllare il mondo fisico in modo semplice e affidabile.

È utile quando servono:

- accesso diretto ai pin di ingresso e uscita
- gestione precisa del tempo
- comportamento prevedibile
- basso consumo
- robustezza
- semplicità costruttiva

Esempi:

- accendere un LED dopo 100 ms
- leggere un sensore ogni 10 ms
- controllare un motore
- attivare un relè
- gestire un piccolo automatismo

Un PC non è ideale per questi compiti perché il sistema operativo può introdurre ritardi imprevedibili.

1.4 Tempo, determinismo e codice non bloccante

Arduino, ad esempio Arduino Uno, esegue il programma in modo sequenziale, senza sistema operativo e senza multitasking.

Quando si dice che Arduino ha un comportamento deterministico non significa che sia sempre veloce, ma che il comportamento è prevedibile: a parità di codice e condizioni, il tempo di esecuzione sarà lo stesso.

Il tempo di risposta dipende quindi direttamente dal codice scritto. Se nel `loop()` viene inserita un'operazione lunga, ad esempio un ciclo che dura 10 secondi, Arduino resterà occupato per 10 secondi. Durante quel tempo non leggerà ingressi, non aggiornerà uscite e non gestirà altri eventi.

Il determinismo garantisce coerenza nei tempi, non automaticamente reattività.

1.5 Gestione del tempo con `millis()`

La funzione `millis()` restituisce il numero di millisecondi trascorsi dall'avvio della scheda.

Serve per gestire il tempo senza bloccare il programma.

Schema concettuale:

```
se (tempo_attuale - tempo_iniziale >= intervallo)
    eseguire operazione
```

In questo modo il programma continua a girare e può svolgere altre attività mentre “attende”.

Esempio concettuale:

```
if (millis() - last >= 10) {
    last = millis();
    eseguire_un_piccolo_passo();
}
```

1.6 Limite importante di `millis()`

`millis()` evita il blocco durante l'attesa, ma non rende automaticamente non bloccante il codice eseguito.

Esempio:

```
if (millis() - last >= 1000) {
    last = millis();
    operazione_lunga(); // es. ciclo da 10 secondi
}
```

In questo caso Arduino resta comunque occupato per tutta la durata dell'operazione lunga.

Per mantenere la reattività occorre:

- evitare `delay()` prolungati

- evitare operazioni lunghe in un unico blocco
- dividere il lavoro in piccoli passi
- eseguire un piccolo passo a ogni iterazione del loop()

1.7 Arduino e breadboard

Arduino viene spesso usato insieme a una breadboard, cioè una basetta per prototipazione senza saldature.

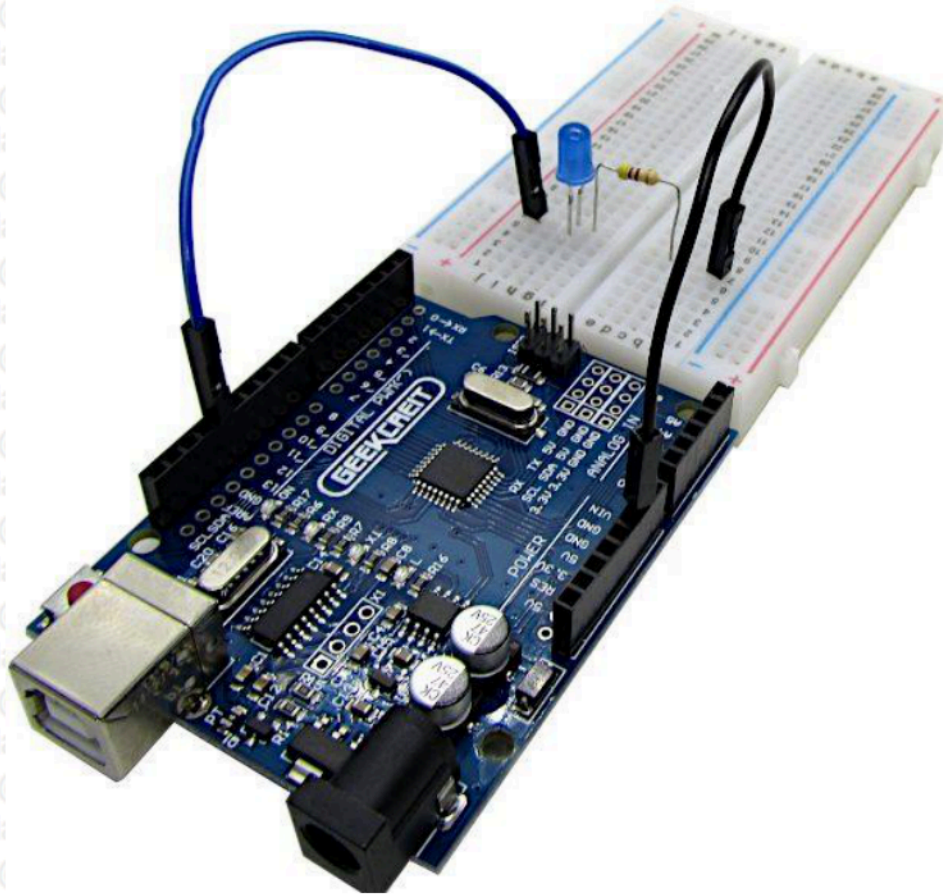


Figure 2: Image

La breadboard permette di collegare temporaneamente:

- sensori
- LED
- pulsanti
- resistenze
- moduli elettronici
- piccoli attuatori

È utile nella didattica perché consente di costruire e modificare rapidamente circuiti sperimentali.

1.8 Quando usare Arduino

Arduino è preferibile quando occorre leggere sensori, controllare attuatori, gestire tempi prevedibili e realizzare sistemi semplici, robusti e a basso consumo.

Casi d'uso tipici:

- termostati
- robotica semplice
- sensori ambientali
- automazione domestica di base
- sistemi embedded industriali semplici
- controllo di luci, motori, relè e piccoli dispositivi fisici

LEZIONE 2 – Raspberry Pi

2.1 Che cos'è Raspberry Pi

Raspberry Pi è un single-board computer, cioè un computer completo realizzato su una singola scheda.

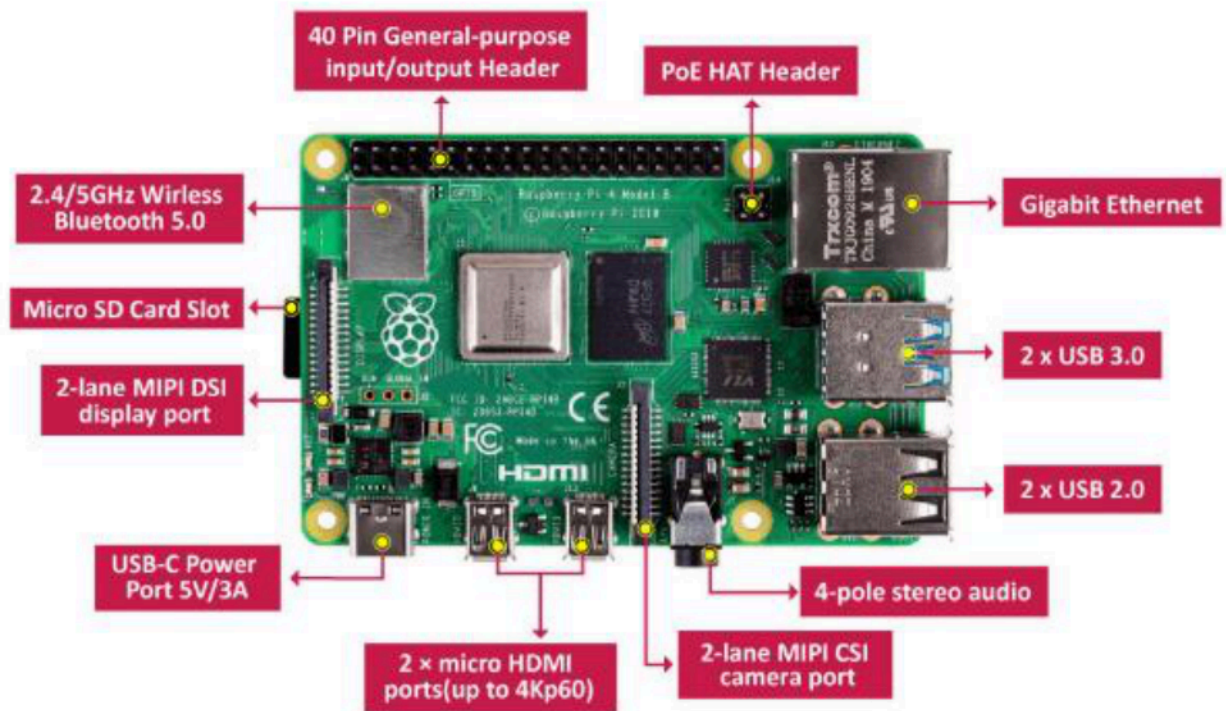


Figure 3: Image

Include:

- CPU
- RAM
- storage, normalmente microSD
- porte USB
- rete
- uscita video
- GPIO per interazione con hardware esterno

La differenza fondamentale rispetto ad Arduino è che Raspberry Pi esegue un sistema operativo, normalmente Linux.

2.2 Raspberry Pi come piccolo computer

Dal punto di vista concettuale Raspberry Pi è un PC, un **single-board computer (SBC)**, cioè un computer **completo** su una singola scheda.



ma:

- * è meno potente di un computer tradizionale
- * è molto più piccolo
- * consuma meno
- * può rimanere acceso a lungo
- * può interagire con hardware esterno tramite GPIO

Raspberry Pi unisce due mondi:

- il mondo dei PC, perché può eseguire software complesso
- il mondo embedded, perché può comunicare con sensori e dispositivi fisici

2.3 Cosa può fare Raspberry Pi

Raspberry Pi può:

- eseguire server web
- gestire database
- controllare sensori
- comunicare in rete
- eseguire servizi Linux
- raccogliere dati
- fornire dashboard e interfacce web
- funzionare come gateway di rete o gateway IoT

2.4 Quando usare Raspberry Pi

Raspberry Pi è preferibile quando serve:

- elaborazione dati
- rete
- interfaccia utente
- server web
- sistema sempre acceso
- basso consumo
- dimensioni ridotte
- integrazione con hardware

Casi d'uso tipici:

- server web leggero
- sistemi IoT
- videosorveglianza
- gateway di rete

- automazione avanzata
- sistemi di controllo con interfaccia web
- dashboard di monitoraggio

LEZIONE 3 – Confronto tra Arduino e Raspberry Pi

3.1 Differenza concettuale fondamentale

Arduino è un dispositivo di controllo orientato all'hardware.

Raspberry Pi è un computer completo orientato al software.

In sintesi:

- Arduino controlla direttamente il mondo fisico
- Raspberry Pi gestisce logica, servizi, rete e interfacce

3.2 Sistema operativo

Arduino:

- non ha sistema operativo
- esegue direttamente il codice caricato
- non ha multitasking reale

Raspberry Pi:

- usa Linux
- supporta multitasking
- può eseguire più servizi contemporaneamente

3.3 Tempo e determinismo

Arduino:

- ha comportamento prevedibile
- è adatto a controllo diretto e tempi precisi
- è più indicato per attività real-time semplici

Raspberry Pi:

- non è deterministico nello stesso modo
- il sistema operativo può introdurre ritardi
- è meno adatto al controllo real-time diretto

3.4 Interazione con hardware

Arduino interagisce con l'hardware in modo diretto, preciso e immediato.

Raspberry Pi può interagire con l'hardware, ma l'accesso è mediato dal sistema operativo. Per questo è meno preciso nei tempi.

3.5 Complessità

Arduino è più semplice:

- codice lineare
- meno configurazione
- meno servizi da gestire

Raspberry Pi è più complesso:

- sistema operativo
- configurazione di rete

- servizi
- sicurezza
- aggiornamenti
- gestione utenti e processi

3.6 Tabella di confronto

Aspetto	Arduino	Raspberry Pi
Tipo di dispositivo	Scheda con microcontrollore	Single-board computer
Sistema operativo	No	Sì, Linux
Programmi contemporanei	No	Sì
Controllo hardware	Diretto	Mediato dal sistema operativo
Tempi prevedibili	Sì	Meno
Rete	Di solito tramite moduli esterni	Integrata nei modelli comuni
Server web	Solo minimale	Completo
Database	Non adatto	Possibile
Consumo	Molto basso	Basso, ma maggiore di Arduino
Uso principale	Sensori e attuatori	Rete, servizi, elaborazione

LEZIONE 4 – Uso combinato di Arduino e Raspberry Pi

4.1 Perché usare entrambi

Molti sistemi reali usano Arduino e Raspberry Pi insieme.

Schema tipico:

Sensori → Arduino → Raspberry Pi → Server / Cloud

Arduino si occupa dell'acquisizione dei dati e del controllo fisico.

Raspberry Pi si occupa dell'elaborazione, della memorizzazione, della comunicazione in rete e dell'interfaccia utente.

4.2 Motivazione tecnica

La separazione è utile perché:

- Arduino è più adatto al controllo fisico diretto
- Raspberry Pi è più adatto a rete, software e servizi
- il sistema diventa più ordinato
- l'architettura è più scalabile
- ogni dispositivo svolge il ruolo per cui è più adatto

Insieme formano un sistema completo ed efficiente.

4.3 Esempi di uso combinato

Monitoraggio sanitario:

- Arduino legge sensori di temperatura, battito o altri parametri
- Raspberry Pi invia i dati a un data-center o a un server

Smart building:

- Arduino controlla luci, sensori e porte
- Raspberry Pi gestisce interfaccia web e log degli eventi

Sistema industriale:

- Arduino controlla macchine o segnali fisici
- Raspberry Pi raccoglie dati e svolge analisi

Rete IoT distribuita:

- molti nodi Arduino o ESP32 raccolgono dati
- Raspberry Pi funziona da gateway centrale

LEZIONE 5 – Connettività di rete

5.1 Concetto generale

Nei sistemi moderni, come IoT, automazione e industria 4.0, la connettività di rete è fondamentale.

Permette di:

- trasmettere dati a server remoti
- integrare dispositivi in sistemi distribuiti
- gestire monitoraggio remoto
- gestire controllo remoto
- collegare sensori, gateway e cloud

Differenza chiave:

- Arduino ha rete opzionale, spesso tramite moduli esterni
- Raspberry Pi ha rete nativa, integrata nei modelli comuni

5.2 Connettività di rete di Arduino

Arduino classico non include quasi mai connettività di rete integrata, anche se esistono eccezioni moderne.

Per collegarlo alla rete servono moduli o shield hardware.

5.3 Arduino con Ethernet

Per la rete cablata si usano:

- Ethernet Shield W5100 / W5500
- moduli economici ENC28J60

Gli shield W5100 / W5500 sono generalmente più stabili e diffusi rispetto ai moduli economici.

Caratteristiche:

- connessione RJ45
- uso di TCP/IP
- librerie Arduino come Ethernet.h

Uso tipico:

- Arduino come client HTTP
- Arduino come piccolo server web minimale

Limiti:

- memoria molto ridotta
- gestione semplificata della rete
- non adatto a server complessi

5.4 Arduino con Wi-Fi

Soluzioni diffuse:

- ESP8266
- ESP32
- vecchi WiFi Shield, oggi meno comuni

Oggi spesso si evita Arduino classico con Wi-Fi shield e si usa direttamente un microcontrollore con Wi-Fi integrato, soprattutto ESP8266 o ESP32.

ESP32 è oggi molto usato perché integra:

- microcontrollore
- Wi-Fi
- Bluetooth/BLE

- buone prestazioni
- basso costo

Supporta protocolli come:

- TCP/IP
- HTTP
- MQTT

5.5 Altri protocolli usabili con Arduino

Arduino può usare anche altri protocolli o tecnologie di comunicazione:

- Bluetooth, ad esempio HC-05
- BLE
- ZigBee, ad esempio XBee
- LoRa, per reti a lungo raggio e basso consumo

Uso tipico:

- sensori distribuiti
- reti IoT a basso consumo
- dispositivi periferici alimentati a batteria

5.6 Limiti tecnici di Arduino in rete

Arduino è adatto a:

- inviare dati semplici
- ricevere comandi
- svolgere il ruolo di nodo periferico

Arduino non è adatto a:

- server web complessi
- database
- servizi multiutente
- elaborazioni pesanti
- gestione avanzata di HTTPS/TLS

I limiti principali sono:

- RAM molto limitata
- memoria ridotta
- gestione semplificata delle connessioni
- difficoltà con protocolli complessi

5.7 Connettività di rete di Raspberry Pi

Raspberry Pi integra già le principali funzioni di rete.

Nei modelli standard sono normalmente presenti:

- porta Ethernet
- Wi-Fi integrato nei modelli recenti
- Bluetooth

Poiché esegue Linux, Raspberry Pi ha uno stack TCP/IP completo.

5.8 Raspberry Pi con Ethernet

La connessione Ethernet consente di usare Raspberry Pi come nodo di rete stabile.

Caratteristiche:

- interfaccia di rete completa
- supporto nativo Linux
- configurazione tramite strumenti di sistema

Comandi e strumenti tipici:

`ifconfig`

ip addr
dhclient

Usi tipici:

- server web con Apache o Nginx
- gateway di rete
- nodo IoT stabile
- server locale

5.9 Raspberry Pi con Wi-Fi

Raspberry Pi 3 e successivi includono Wi-Fi integrato.

La configurazione può essere gestita dal sistema operativo e, in alcuni casi, tramite file come:

`/etc/wpa_supplicant/wpa_supplicant.conf`

Raspberry Pi supporta normalmente:

- DHCP
- DNS
- routing
- VPN
- servizi di rete Linux

5.10 Capacità avanzate di rete di Raspberry Pi

Raspberry Pi può funzionare come:

- server web completo
- server database, ad esempio MySQL o PostgreSQL
- reverse proxy
- firewall con iptables o nftables
- gateway IoT
- broker MQTT, ad esempio Mosquitto
- nodo di raccolta dati
- sistema di dashboard

Questo è possibile perché esegue Linux e dispone di uno stack di rete completo.

5.11 Confronto rete Arduino / Raspberry Pi

Caratteristica	Arduino	Raspberry Pi
Rete integrata	No, di base	Sì, nei modelli comuni
Complessità rete	Bassa	Alta
Protocolli supportati	Limitati	Completi
Server web	Molto semplice	Completo
HTTPS/TLS	Limitato o complesso	Gestibile
Database	Non adatto	Possibile
Ruolo tipico	Nodo periferico	Nodo centrale / gateway

Sintesi:

- Arduino è un nodo semplice di rete
- Raspberry Pi è un nodo intelligente di rete

LEZIONE 6 – Architetture reali e casi da esame

6.1 Architettura IoT classica

Schema tipico:

Sensori → Arduino / ESP32 → Raspberry Pi → Internet → Cloud

Ruoli:

Arduino o ESP32:

- acquisizione dati
- lettura sensori
- controllo attuatori
- invio dati via seriale, Wi-Fi o altro protocollo

Raspberry Pi:

- aggregazione dati
- normalizzazione dati
- invio al cloud
- dashboard web
- database locale
- eventuale broker MQTT

6.2 Raspberry Pi come gateway IoT

Raspberry Pi può funzionare da gateway IoT.

Funzioni principali:

- raccoglie dati da più Arduino o ESP32
- normalizza i dati
- memorizza dati localmente
- invia dati a un server remoto
- espone una dashboard
- gestisce comunicazioni con il cloud

Protocolli tipici:

- MQTT
- HTTP REST
- WebSocket

6.3 Caso da esame: monitoraggio sanitario

Scenario:

Sistema di rilevazione di parametri di un paziente.

Soluzione:

- Arduino legge sensori, ad esempio temperatura o battito
- Raspberry Pi raccoglie i dati
- Raspberry Pi invia i dati a un data-center o a un server

Motivazione:

- separazione tra acquisizione e comunicazione
- maggiore affidabilità
- architettura più chiara

6.4 Caso da esame: smart building

Scenario:

Edificio intelligente con controllo di luci, porte e sensori.

Soluzione:

- Arduino controlla luci, sensori e porte
- Raspberry Pi gestisce interfaccia web, log eventi e rete

Motivazione:

- Arduino gestisce il controllo fisico
- Raspberry Pi gestisce software, rete e monitoraggio

6.5 Caso da esame: sistema industriale

Scenario:

Sistema di controllo e monitoraggio in ambiente industriale.

Soluzione:

- Arduino controlla segnali, sensori o macchine
- Raspberry Pi raccoglie dati e svolge analisi

Motivazione:

- il controllo diretto resta su microcontrollore
- l'elaborazione e la raccolta dati sono affidate a Raspberry Pi

6.6 Caso da esame: rete IoT distribuita

Scenario:

Sistema di monitoraggio ambientale distribuito.

Soluzione:

Nodi periferici:

- Arduino o ESP32
- sensori
- comunicazione Wi-Fi, LoRa, ZigBee o altro protocollo

Nodo centrale:

- Raspberry Pi

Funzioni del Raspberry Pi:

- server MQTT
- database locale
- dashboard web
- invio dati al cloud

Motivazione tecnica:

- separazione dei livelli
- scalabilità
- affidabilità
- migliore organizzazione della rete

6.7 Add-on e scelte tecnologiche attuali

In passato era comune usare Arduino con shield Ethernet o Wi-Fi.

Oggi, per molti progetti con rete, è più comune usare:

- ESP32 come microcontrollore con Wi-Fi integrato
- Raspberry Pi come nodo centrale o gateway

Situazione attuale:

- Arduino classico resta utile per controllo semplice e didattica
- ESP32 sostituisce spesso Arduino nei progetti IoT con Wi-Fi
- Raspberry Pi resta adatto per rete, servizi e gestione centralizzata

6.8 Errori tipici da evitare nelle tracce d'esame

Errore 1:

Usare Arduino come server web complesso.

Problema:

Arduino non ha risorse sufficienti per gestire servizi complessi.

Soluzione:

Usare Arduino o ESP32 come nodo periferico e Raspberry Pi come server o gateway.

Errore 2:

Usare Raspberry Pi per controllo real-time diretto.

Problema:

Il sistema operativo può introdurre ritardi non prevedibili.

Soluzione:

Usare Arduino per il controllo diretto e Raspberry Pi per elaborazione e rete.

Errore 3:

Non separare acquisizione e rete.

Problema:

L'architettura diventa poco scalabile e poco chiara.

Soluzione:

Separare i ruoli:

- Arduino / ESP32 → acquisizione dati e controllo fisico
- Raspberry Pi → rete, servizi, gateway, dashboard e invio al cloud

CONCLUSIONE

Arduino e Raspberry Pi non sono alternative equivalenti.

Sono strumenti diversi, progettati per problemi diversi.

Arduino è adatto al mondo fisico:

- sensori
- attuatori
- controllo diretto
- tempi prevedibili
- basso consumo
- codice semplice

Raspberry Pi è adatto al mondo logico e di rete:

- Linux
- servizi
- database
- server web
- dashboard
- gateway
- comunicazione con cloud

Nei sistemi reali spesso si usano entrambi:

sensori → microcontrollori → gateway → cloud

La rete è normalmente gestita dal Raspberry Pi, mentre i dati sono prodotti da Arduino, ESP8266 o ESP32.

Ricordare che anche in questo caso per una traccia d'esame è necessario:

- scegliere il dispositivo corretto
- assegnare a ogni componente il ruolo più adatto
- separare acquisizione, elaborazione e comunicazione
- e soprattutto
- **motivare tecnicamente la scelta**

Solitamente nelle griglie di valutazione una voce delle 4-5 è dedicata all'argomentazione